19. Tree ADT -I

T.V. GEETHA

Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of an important Linear ADTs. In this module we will discuss Trees – an important hierarchical data structure.

Learning Objectives

The learning objectives of the module are as follows:

- To explain the concept of Trees
- To discuss General Trees with some examples
- To describe the representations of General Trees
- To explain the different tree traversals
- To discuss the conversion of General Trees to Binary Trees

19.1 Introduction

So far in the previous modules we have discussed linear data structures such as lists, stacks and queues. We also have other types of data structures which are non-linear, for example trees and graphs. Trees are hierarchical data structures. Trees are used for applications such as sorting, searching, expression evaluation, etc. Trees are especially well suited to recursive algorithm implementations.

19.1.1 What are Trees?

A tree consists of a specially designated node called the root and zero or more sub-trees. A tree can also be empty containing no nodes. Figure 19.1 shows an example

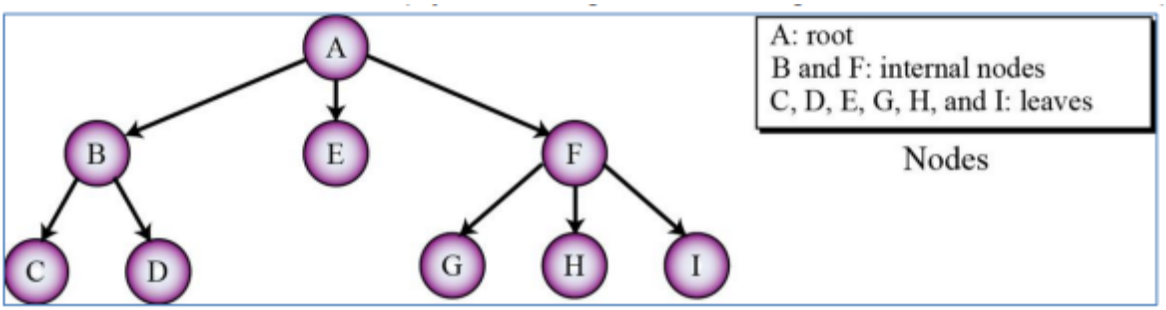


Figure 19.1 Example of a Tree

of a tree where A is the root of the tree, B and F are internal nodes that is they have children and C,D,E,G, H and I are all leaf nodes (that is they have no children.

19.1.2 **Trees -Terminology**

Generally trees can be used to represent relationships. Let us look at some terminologies associated with trees. Trees are hierarchical in nature and **"Parent-child"** relationship exists between nodes in a tree. This concept can be generalized as **ancestor and descendant** relations. Lines between the nodes are called edges. **Ancestors** of a node are it's parent, grandparent, great-grandparent, etc. Similarly **descendant** of a node are it's child, grandchild, great-grandchild, etc. **A sub-tree** in a tree is any node in the tree together with all of its descendants. **Depth** of a node is the number of ancestors it has. **Height** of a tree is the maximum depth of any of it's

node. **Degree** of a node is the number of its children the node has. On the other hand the d**egree** of a tree is the maximum degree of it's nodes.

Let us consider the example given in Figure 19.2, the r**oot** is the node without a parent (A), **siblings** are nodes that share the same parent, example are nodes E and F that share the same parent B. **Internal nodes** are nodes with at least one child (example – A, B, C, F), while **external node** (or leaf nodes) are nodes without children (example -E, I, J, K, G, H, D). A **subtree** is a tree consisting of a node and its descendants (example subtree rooted at C and having descendants G and H). Each node in a tree may have a subtree. In other words, the subtree of each node includes one of its children and all descendants of that child (Figure 19.3).

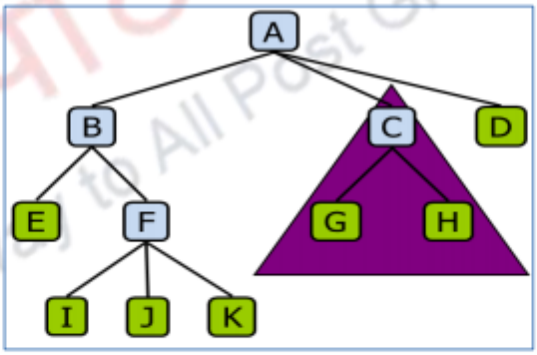


Figure 19.2 Another Example of a Tree

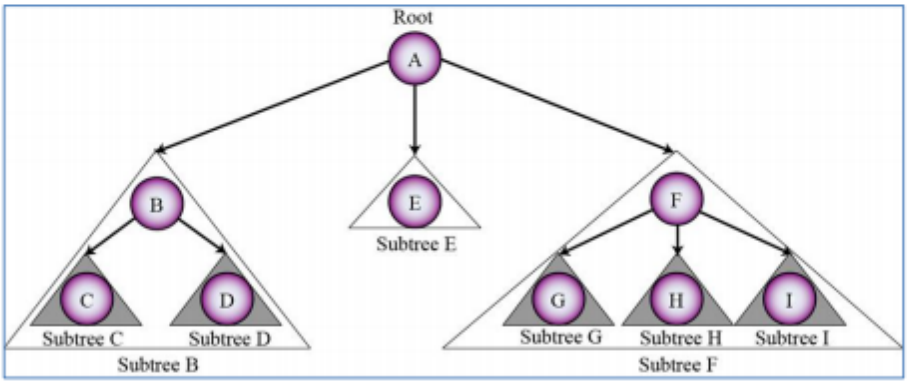


Figure 19.3 Subtrees of a Tree

To summarize we can divide the vertices of a tree into three categories: the *root*, *leaves* and the *internal nodes*. The root has no parent and can have zero or more children, the leaf has one parent and no children while the internal nodes has parent and one or more children.

19.2 Kinds of Trees

One of the classifications of trees is based on the maximum number of children a tree are allowed to have. General trees are trees where a node can have any number of children. N-ary trees are trees where a node can have a maximum of N children. Binary trees are 2-ary trees where a node can have a maximum of two children.

19.2.1 General Tree

The general tree has a specially designated node r, called the root of the tree and sets that are general trees, called subtrees of r. Here there is no restriction on the number of subtrees a node can have.

19.2.2 N-ary Tree

The n -ary tree can be defined as a set T of nodes that is either empty or partitioned into disjoint subsets. It has a specially designated node r, called the root of the tree and n possibly empty sets that are n-ary subtrees of r.

19.2.3 Binary Tree

The binary tree can be defined as a set T of nodes that is either empty or partitioned into two disjoint subsets. It has a specially designated node r, called the root of the tree and two possibly empty subtrees called as left and right subtrees of r.

19.3 Representation of Trees

The concept of a tree is verycommon as well as very important. Each node in the tree has only have only one parent, and a node has children. Of course leaf nodeshave no children and the root node which is at the top has no parent. We need to look at effective data structures to represent trees.

19.4 Importance of Trees

Trees are important since execution and processing for many applications can be

expressed as a tree, e.g. method calls as a program runs, searching a maze or puzzle, etc.. Trees are important for cognition and computation in areas such as computer science, language processing (by humans or computers) for example parse trees and knowledge representation (or modeling of the "real world") for example family trees; biological taxonomy (kingdom, phylum, ..., species); etc. Let us now look at some examples of trees.

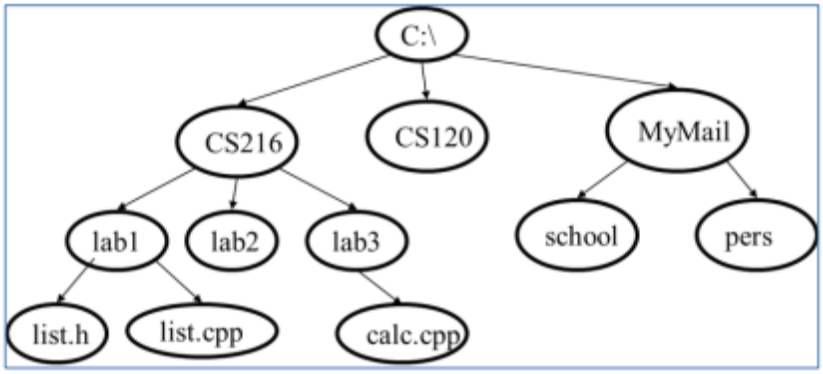


Figure 19.4 Tree Example: File System

Figure 19.4 shows an example of representing a file system organization of a computing system using trees. We can see that the file system is naturally represented using the hierarchical tree data structure. This example shows a 3-ary tree. Figure 19.5 shows the representation of semi-structured documents using trees.In both HTML and XML elements are identified in a document by their **starttag** and **endtag**. Complex elements are constructed from other elements hierarchically, whereas simple elements contain data values. We can see the correspondence between the HTML/XML textual representation and the tree structure. HTML elements are represented by tree nodes and organized into a hierarchy. In the tree representation, internal nodes represent complex elements, whereas leaf nodes represent simple elements.

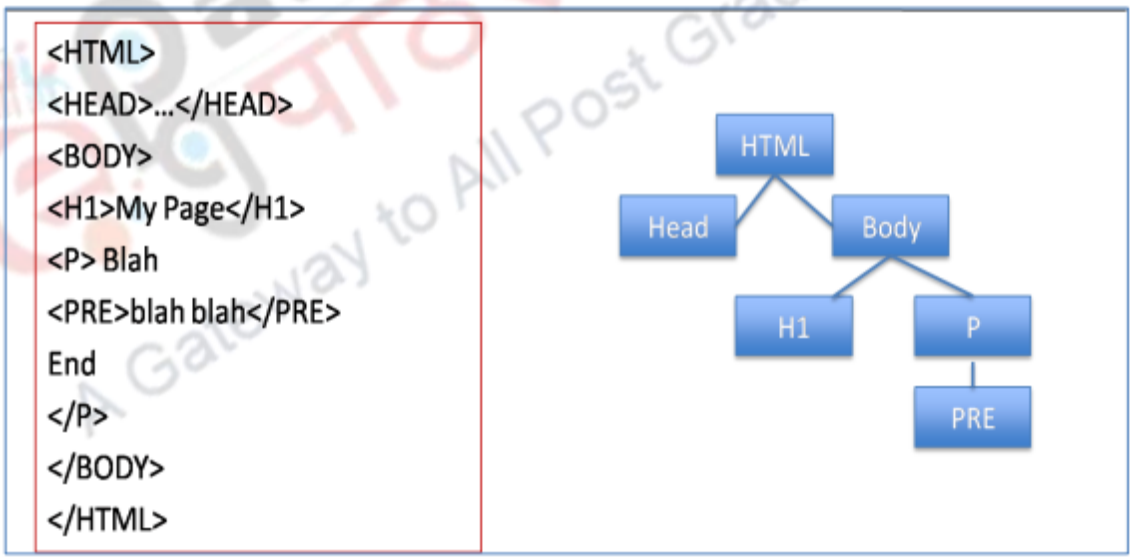


Figure 19.5 Tree Example: XML and HTML documents

In this representation, tags are associated with the edges which represent the schema names that is the names of attributes, object types (or entity types or classes), and relationships. The internal nodes represent individual objects or composite attributesand the leaf nodes represent actual data values of simple (atomic) attributes.

19.5 Recursive Data Structure

In general a recursive data structure is a data structure that contains a pointer or reference to an instance of itself. The tree data structure can be expressed as a recursive data structure. Figure 19.6 shows the recursive definition of the binary tree.Recursion is a *natural* way to express many algorithms.For recursive data-structures, recursive algorithms are a natural choice.

```
public class TreeNode<T>

{
T  nodeItem; TreeNode<T> left, right; TreeNode<T> parent;
...
}
```

Figure 19.6Tree as a Recursive Data Structure

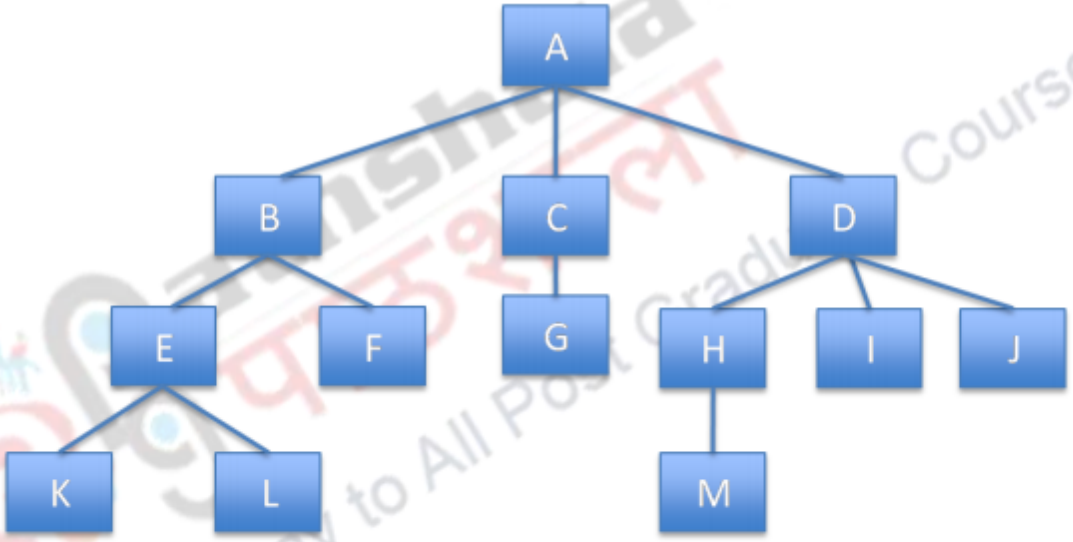
19.6 Tree ADT

In our discussion on trees as an ADT we use positions to abstract nodes. Generic functions include:

• integer**size**() – This function gives the number of nodes in the tree.
• boolean**isEmpty**() – This Boolean function signals true if the tree is empty
• Iterator **elements**() – This function outputs all the elements of the tree in an iterative manner
• Accessor functions: Some of the important functions to access the tree are:
– position **root**() – This function outputs the position of the root
– position **parent**(p) – This function outputs the position of the parent of a node
– positionIterator**children**(p) – This function outputs the positions of all the children of a node
• Query functions: These are some of the functions to query the tree data structure
– boolean**isInternal**(p) – This Boolean function finds if a given node is an internal node.
– boolean**isExternal**(p) -This Boolean function finds if a given node is an external or leaf node.
– boolean**isRoot**(p) – This Boolean function finds if a given node is a root node.
• Update functions: Some of the functions to update the tree are given below. Additional update functions may be defined by data structures implementing the Tree ADT
– **swapElements**(p, q) – This functions swaps two elements of the tree given the tree T.
– **replaceElement**(p, e) – This functions replaces an element of the tree given the tree T.

19.7 Representation of the Tree

Before we discuss the details of the representation of the tree, let look at intuitive representation of Tree node. The tree can be represented by a list (**Figure 19.7**).



- List Representation
  - (A (B (E (K, L), F), C (G), D (H (M), I, J)))

Figure 19.7 Representation of the Tree as a List

Here the root comes first, followed by a list of links to sub-trees

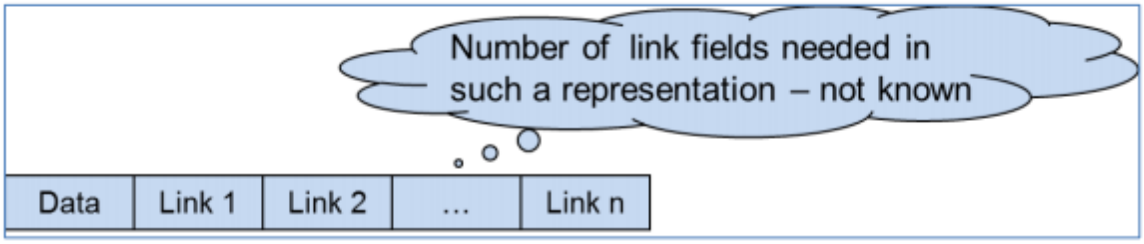


Figure 19.8 (a) An Intuitive Representation of the Tree Node

Every tree node in general, how many ever children it can have contains data which stores useful information and pointers to its children and these nodes can be used for the representation of the general tree (Figure 19.8 (a) and 19.8 (b)). However here we do not know the number of link fields needed since the number of children of each node is unknown.

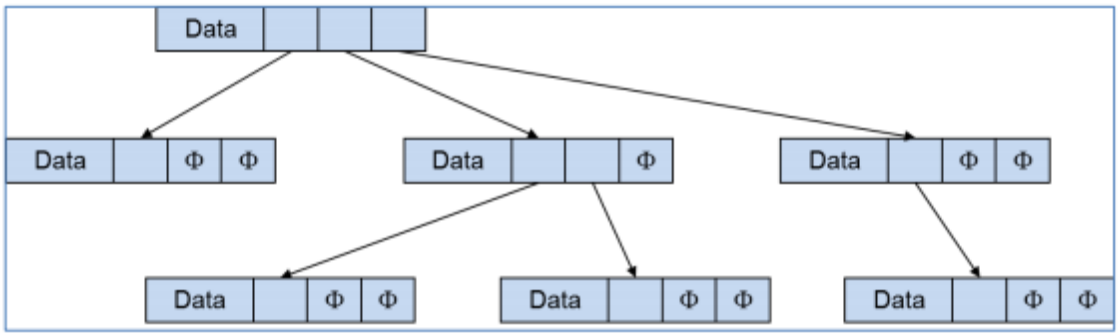


Figure 19.8 (b) A Representation of the General Tree

Therefore the node of the general tree can be represented by a node storing the data, pointer to the parent node and information about where the sequence of children nodes are stored (Figure 19.9 (a)). Now let us look at different ways in which general trees can be represented.

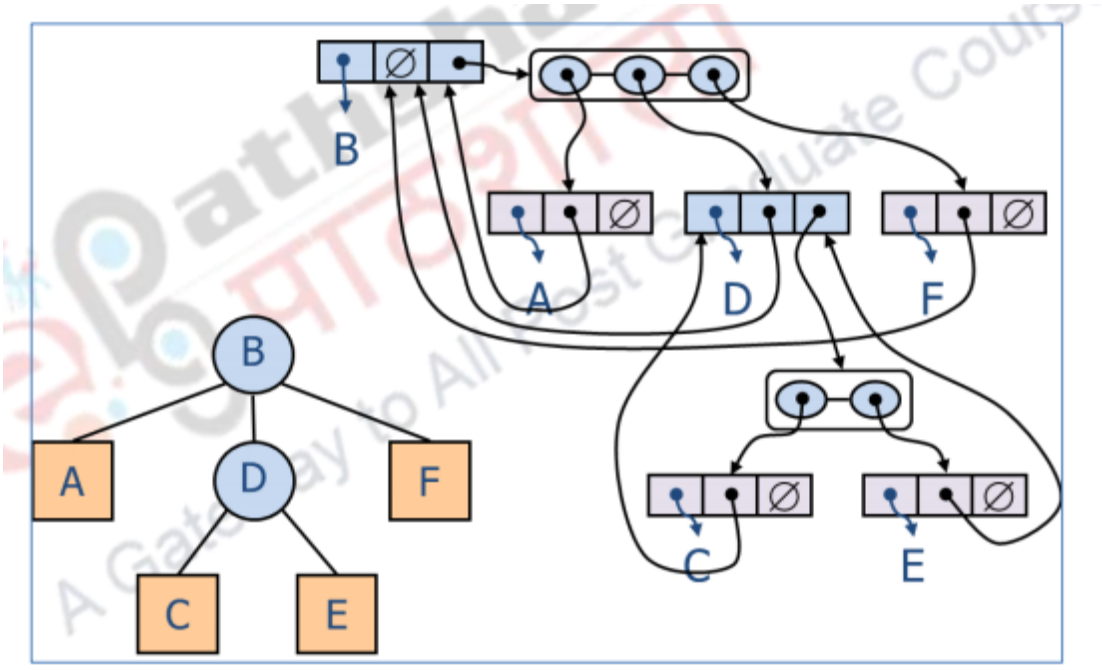


Figure 19.9 Representation of the General Tree

19.7.1 General Trees –Implementation

One way to implement a general tree is to use a node whose structure uses a node consisting of the data element, pointer to the parent node and a pointer to the sequence of children nodes (Figure 19.9). In another link representation, specifically, a node left pointer points to its left-most child and it's right pointer points to a linked list of nodes that are it's siblings. Figure 19.10 shows the Left Child, Right Sibling Representationwhere we put the left child node to the node's left link and put the sibling to the node's right link.

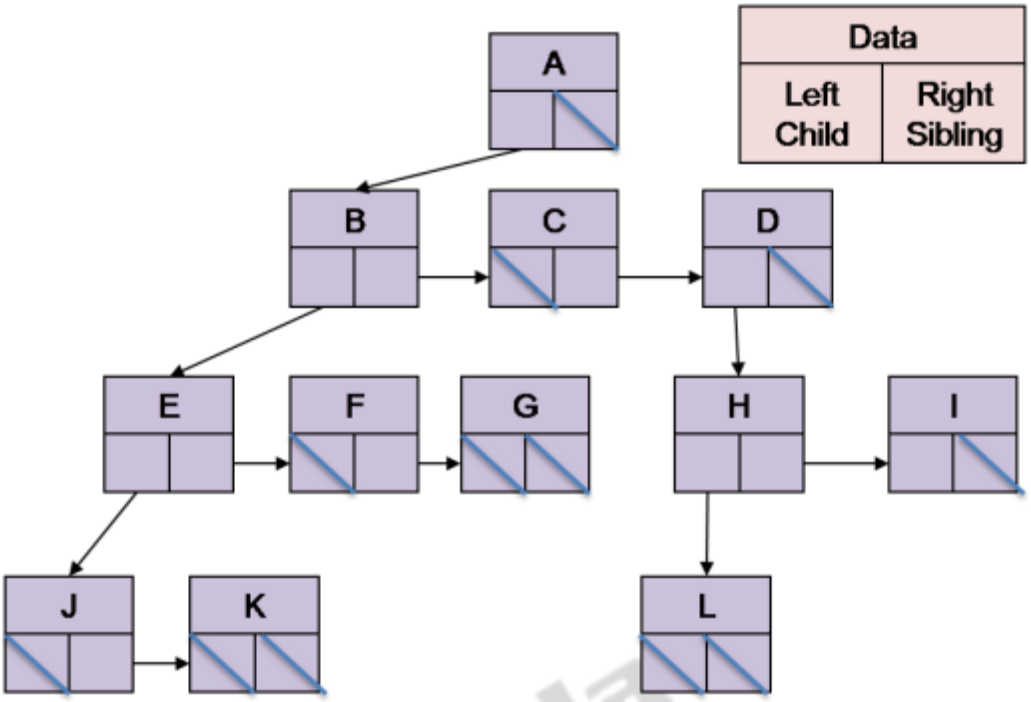


Figure 19.10 Left Child, Right Sibling Representation

19.7.2 Parent Pointer Implementation

Figure 19.11 shows the parent pointer representation where we have an array where each element of the array stores the label of the node and the index to it's parent. For example the node C is stored in location of the array indexed 3, and its parent's index 1 which is the location of C's parent A.

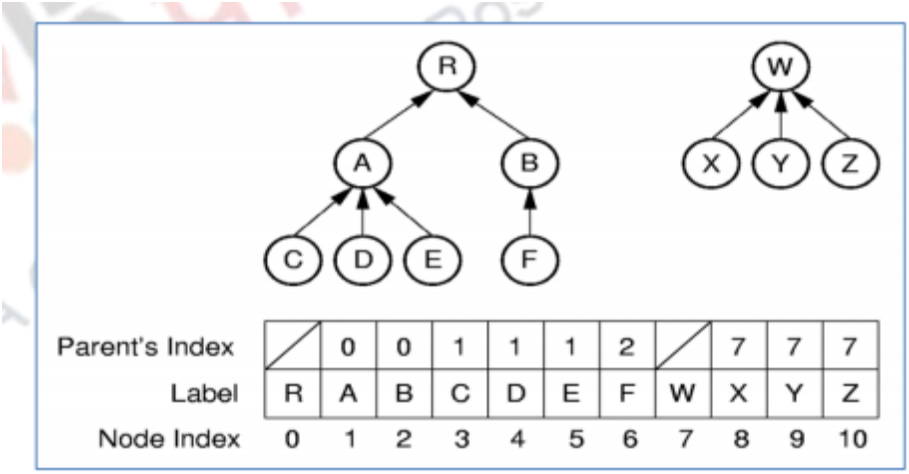


Figure 19.11 Parent Pointer Representation

19.8 Tree Traversal

An important operation associated with trees is traversal of a tree. A traversal visits the nodes of a tree in a systematic manner. Depending on the order in which the nodes are traversed, there are three main types of traversal namely inorder, preorder and postorder traversals. These traversals can be defined recursively as given below:

19.8.1 Inorder Traversal

In inorder traversal, the left most subtree of a node is visited in inorder, then the node is visited and then all the other subtrees are visited inorder. It is recursively as follows: Traverse in inorder the left subtree, visit the root and finally traverse in inorder the right subtree in case the tree has only two children. The algorithm for the inorder traversal of a general tree is given in Figure 19.12 where we traverse in inorder the left subtree, visit the root and finally traverse in inorder all children other than the leftmost subtree. Figure 19.13 shows the example for the Inorder traversal of a general tree.

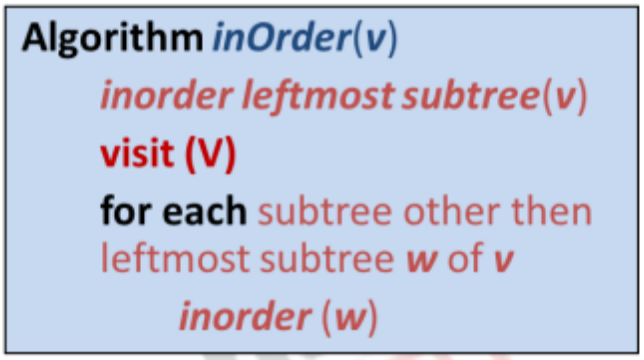


Figure 19.12 Inorder Traversal of a General Tree

In this traversalinorder(*Become Rich*) we start at the root node*Become Rich*, but we need to inorder traverse its Left subtree. So we call inorder(*Motivations*).Here again we need to inorder traverse its Left subtree. So we call inorder(*Enjoy Life*). At this point Left subtree is empty so we go to the next step of the Inorder Traversal with *Enjoy Life*which is visit node, so we visit this node ***Enjoy Life***. Now this node has no Right subtrees and hence this call is completed. This essentially means that the Inorder Left subtree call by inorder(*Motivations*)is completed, so we visit this node, therefore the second node to be visited is ***Motivations.***Now we go to the third step in the call inorder(*Motivations*)which is inorder traversal of its children except leftsubtree. Therefore we call inorder(*Help Poor Friends*). Since this node has no children, only step 2 takes place that is ***Help Poor Friends*** is the third node to be visited. Now we have completed the first step of inorder(*Become Rich*)and so we go step to and ***Become Rich***is the fourth node to be visited. The process continues by traversing the other children of *Become Rich*and the nodes as visited in the order shown in Figure 19.13.

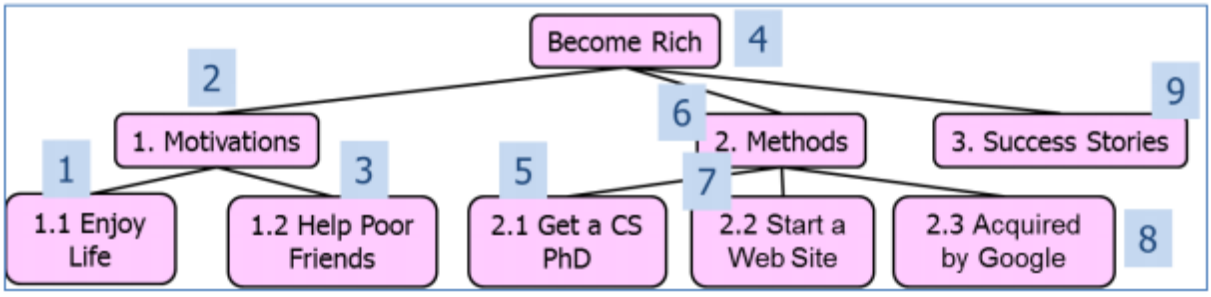


Figure 19.13 Example of the Inorder Traversal of a General Tree

19.8.2 Preorder Traversal

In preorder traversal we first visit the root and then traverse in preorder the children (subtrees). In other words in a preorder traversal, a node is visited before its descendants. A typical application of such a traversal is the printing of a structured document. The algorithm for the preorder traversal of a general tree is given in Figure 19.14 where we first visit the root, then traverse in inorderall the children of the root node. Figure 19.15 shows the example for the Preorder traversal of a general tree.

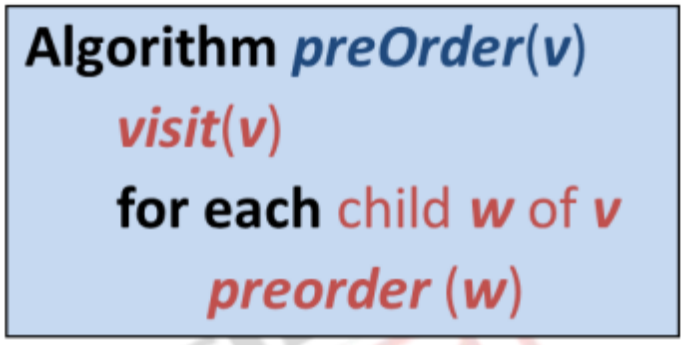


Figure 19.14 Preorder Traversal of a General Tree

In this traversal preorder(*Become Rich*) we start at the root node *Become Rich*, where we first visit this node, so ***Become Rich***is thefirst node to be visited. Next we need to preorder traverse its Left subtree. So we call preorder(*Motivations*).Here again we need to first visit the root node of this Left Subtree, so the second node to be visited is ***Motivations***.Next we need to preorder traverse left subtree of *Motivations*. So we call preorder(*Enjoy Life*).Here again we need to first visit the root node of this Left Subtree, so the third node to be visited is***Enjoy Life***.At this point we see that *Enjoy Life*has no children and hence we have completed the call preorder(*Enjoy Life*).Therefore we go to the next child of preorder(*Motivations*)and call preorder(*Help Poor Friends*).Again here we first visit the node, so the fourth node to be visited is ***Help Poor Friends***.The process continues by traversing the other children of *Become Rich*and the nodes as visited in the order shown in Figure 19.15.

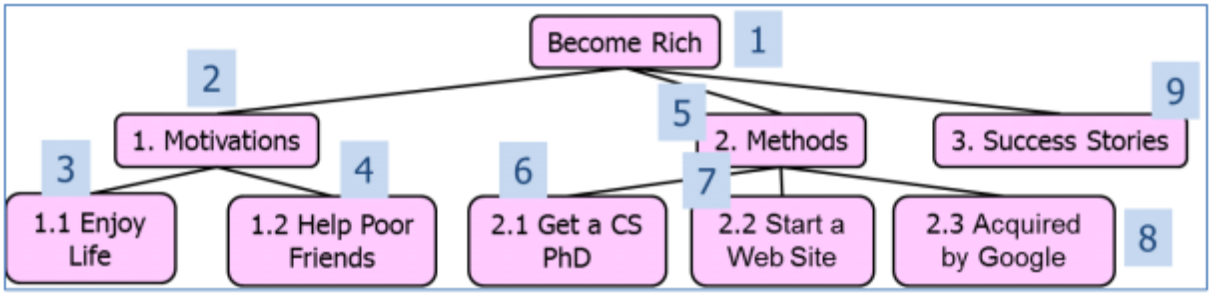


Figure 19.15 Example of the Preorder Traversal of a General Tree

19.8.3 Postorder Traversal

In postorder traversal we first traverse in postorder the children (subtrees) of the root before visiting the root. In other words, in a postorder traversal, a node is visited after its descendants. The organization of files in a directory and its subdirectories for accessing is a typical application of this type of traversal. The algorithm for postorder traversal of the general tree is given in Figure 19.16 where we first visit the root, then traverse in postorder all the children of the root node. Figure 19.17 shows the example for the postorder traversal of a general tree.

```
Algorithm postOrder(v)
    for each child w of v
        postOrder (w)
    visit(v)
```

Figure 19.16 Postorder Traversal of a General Tree

In the example we see that all children of a node are visited before the root is visited. Considering the subtree homeworks/, the node is visited after its children h1c.doc and h1nc.dov have been visited.

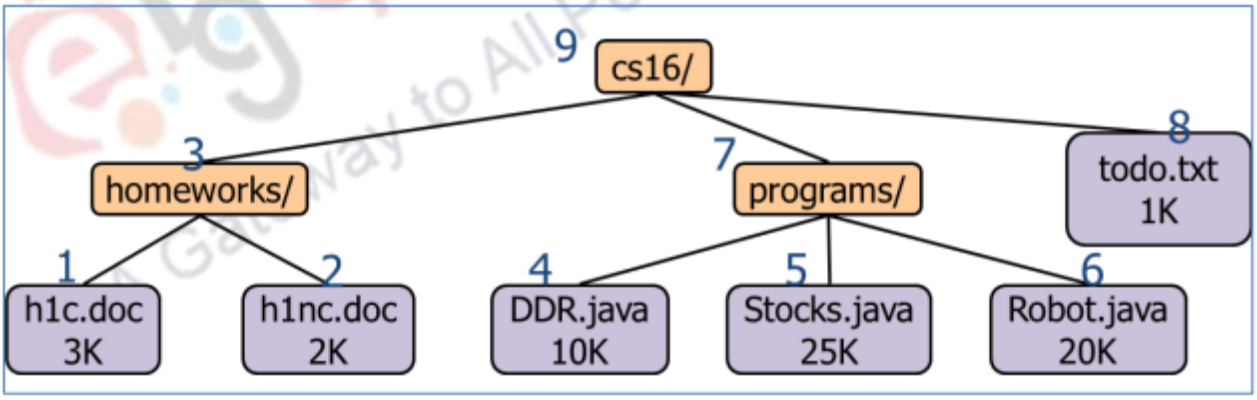


Figure 19.17 Example of the Postorder Traversal of a General Tree

19.9 Problems with General Trees and Solutions

The use of general trees is not very convenient and its implementation is not very efficient. One problem associated with general trees is that the number of references needed for each node must be equal to the maximum that will be used in the tree in some implementations and hence varies as the maximum number of children of a general tree changes. Moreover, most of the algorithms for searching, traversing, adding and deleting nodes havetodeal with the fact where there are not just two possibilities for any node but multiple possibilities. This makes the algorithms more complex. However there are situations that some problems naturally map to general trees. The solution to handling general trees is to convert these general trees to binary trees. In this way the algorithms that are used for binary tree processing can be used with only minor modifications.

19.10 Converting a General Tree to a Binary Tree

Now we will discuss the conversion of a general tree to a binary tree. The steps for this conversion are given below:

1. Use the root of the general tree as the root of the binary tree.
2.Determine the first child of the root. This is the leftmost node in the general tree at the next level.
3.Insert this node. The child reference of the parent node refers to this node.
4.Continue finding the first child of each parent node and insert it below the

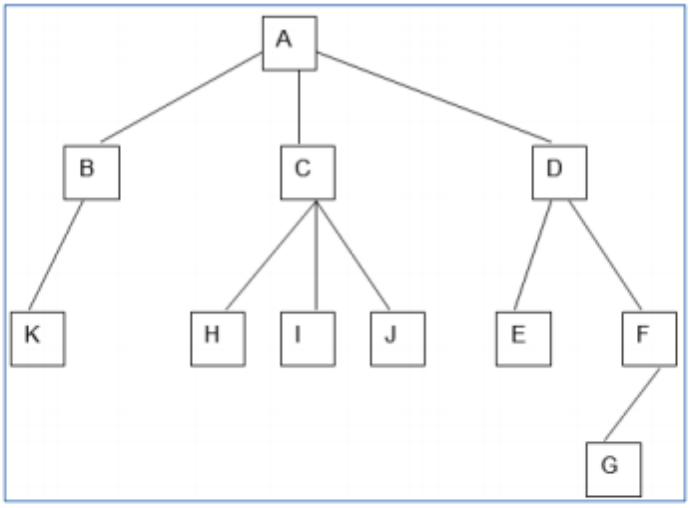
parent node with the child reference of the parent to this node.

5.When no more first children exist in the path just used, move back to the parent of the previous node processed and repeat the above process that is we need to determine the first sibling of the last node encountered.

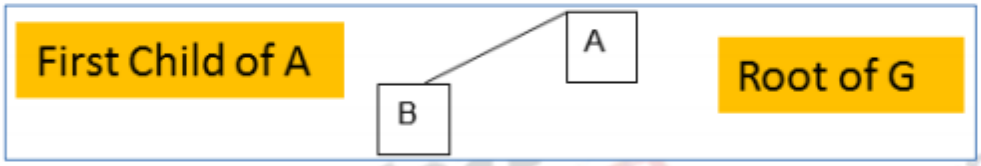
6. Complete the tree for all nodes.

7. For completing the tree, the sibling next to the first child is made its right child and its sibling becomes its right child and so on until all siblings are accounted for.

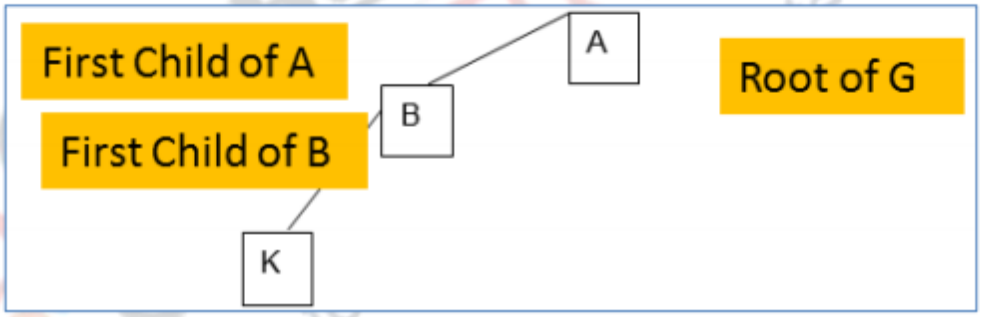
We will illustrate these steps using an example. Figure 19.18 (a-g) shows steps in the conversion of general tree to binary tree. Figure 19.18 (a) shows a general tree. As per step 1 given in the conversion process A, the root of the general tree is also the root of the binary tree and as per step 2 & 3 the first child of A that is B is the left child of A (Figure 19.18 (b)). Again as per step 4 the first child of B that is K is the left child of B (Figure 19.18 (c)). Now we have exhausted the first children in the path A-B-K so we go to step 6 where we go to the last node processed K and see if it has other children. In this example K has no other children and neither does B. So we go back to A and start processing other children of A. Now C, the sibling of B becomes the right child of B, and C's right sibling D becomes it's right child (Figure 19.18 (d)). Now we process node C, it's first child H becomes C's left child, while H's sibling I becomes it's right child and I's sibling becomes it's right child (Figure 19.18 (e)). Now Now we process node D, it's first child E becomes D's left child, while E's sibling F becomes it's right child. F's first child G becomes it'sleft child (Figure 19.18 (f)). Now all nodes of the general tree have been processed and the binary tree corresponding to the general tree has been obtained.

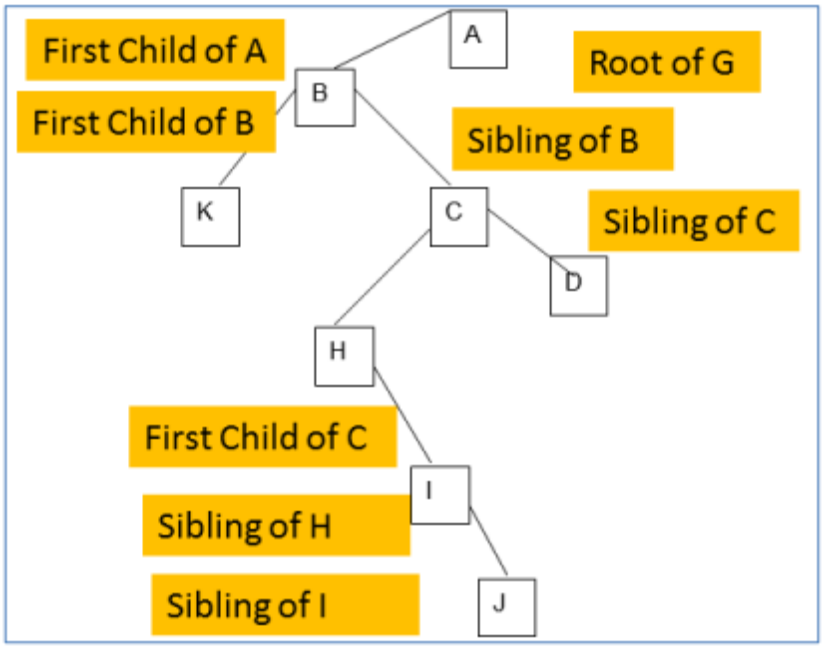


(a)

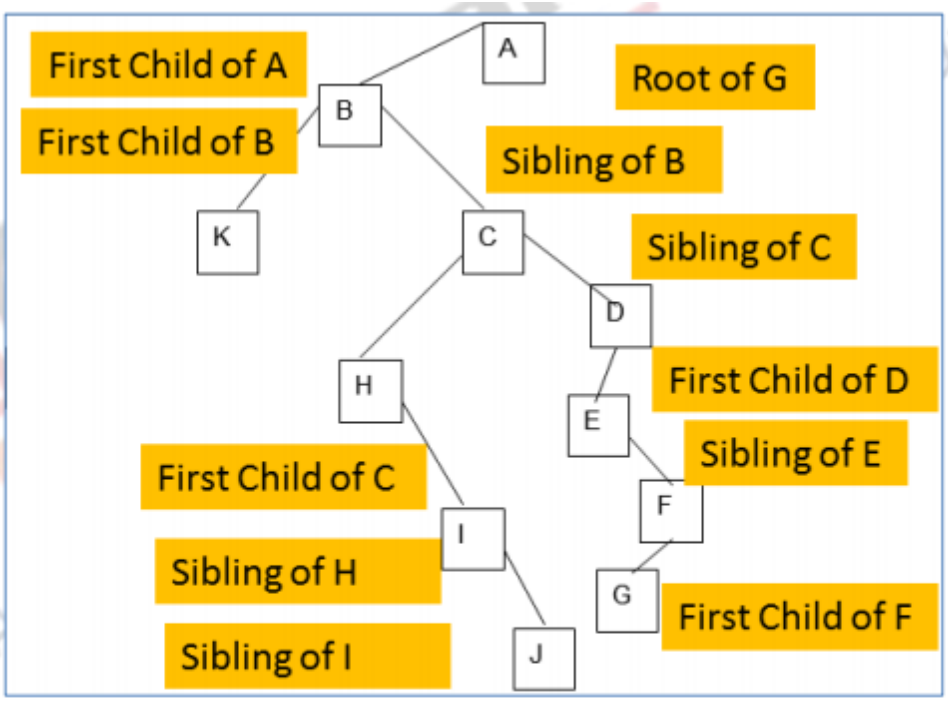


(b)





(e)



(f)

Figure 19.18 Conversion of General Tree to Binary Tree

Summary

- Explained the concept of Trees
- Discussed General Trees with some examples
- Described some representations of General Trees
- Explained the different tree traversals
- Outlined the conversion of General Trees to Binary Trees

---

you can view video on Tree ADT -I 

Web Links
www.cse.unt.edu/~huangyan/3110/Lectures/Trees.ppt
nlg.csie.ntu.edu.tw/courses/Datastructure/chapter5.ppt
www.dsm.fordham.edu/~agw/datastr/slides/Chapter15-Trees.ppt
faculty.mu.edu.sa/download.php?fid=117083
pages.cpsc.ucalgary.ca/.../CPSC335_6_**General**%20and%20Variants.**ppt**
birg.cs.wright.edu/cs400/slides/**General**%20**Trees**.**ppt**
www.rsbauer.com/class/dsa2/slides/**General**%20**Trees**.**pdf**
https://cs.wmich.edu/gupta/teaching/.../lec18%20**trees**%20080805.**ppt**
http://cms.cvrgi.edu.in/uploaddocs/6_Trees
pages.cpsc.ucalgary.ca/~marina/335/B**tree2**.doc
http://cs.nyu.edu/~gottlieb/courses/2000

Supporting & Reference Materials
Adam Drozdek,"Data Structures and Algorithms in C++", Cengage; 4th edition,2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Robert Sedgewick, Kevin Wayne "Algorithms", Addison-Wesley Professional, 4th Edition, 2011
Thomas H. Cormen, et.al, "Introduction to Algorithms"The MIT Press; 3rd edition, 2009
Anany Levitin "Introduction to the Design and Analysis of Algorithms", Pearson; 3rd edition 2011